



Créer et intégrer un Chatbot multimodal dans une application web - partie 2

Table des matières

I - Chatbot : ajout endpoint « <i>Analyse de sentiment</i> »	3
II - Chatbot : ajout endpoint “Image”	19
III - Chatbot : ajout endpoint “Voix”	35
IV - Synthèse	60
V - Auto-évaluation	62
A. Exercice : Quiz	62
B. Exercice : Cas pratique.....	66
Solutions des exercices	67

I Chatbot : ajout endpoint « *Analyse de sentiment* »

Lorsqu'un utilisateur utilise un chatbot, il n'y a rien de plus frustrant que de chercher à être assisté pour un problème spécifique, mais de ne pas avoir la possibilité d'interagir avec un être humain quand la situation l'exige.

C'est la raison pour laquelle nous allons ajouter un vrai endpoint d'analyse de sentiment à notre endpoint chat.

Nous avons déjà implémenté une version basique d'analyse de sentiment, mais nous allons cette fois-ci beaucoup plus loin en utilisant un modèle PyTorch via HuggingFace.

Pourquoi PyTorch ?

Parce que cela permet d'utiliser un pipeline reconnu dans tout l'écosystème IA moderne.

De plus, HuggingFace propose des modèles pré-entraînés efficaces et simples à intégrer. Il s'agit d'une référence dans le domaine du NLP (Natural Language Programmation).

Mais avant de commencer, nous allons faire un petit point vocabulaire.

Lazy Loading, Singleton et Pipeline Huggingface

Az Définition

Le terme Lazy loading fait référence au fait de charger un composant (dans notre exemple un modèle IA) uniquement au moment où il est utilisé et non dès le démarrage. Cela permet d'économiser mémoire et temps de démarrage.

Ensuite, singleton. Singleton nous permet de créer une seule instance d'un objet (ici le pipeline HuggingFace) pour toute l'application et de la réutiliser par la suite.

Enfin, une pipeline HuggingFace est un composant tout en un. Il permet de lancer une tâche d'IA (ici, l'analyse de sentiment) en se basant sur des modèles déjà établis.

Il existe d'autres pipeline HuggingFace, retrouvez-les à l'adresse suivante Hugging Face¹.

Ajout de l'analyse de sentiment avec Pytorch et Hugging Face

Méthode

Nous allons commencer par installer Pytorch et Transformers qui est la bibliothèque de HuggingFace. Elle contient plusieurs modèles pré-entraînés dont notre modèle d'analyse de sentiment.

Tapez les commandes suivantes dans le dossier backend, avec le venv activé :

```
demo@mac cd backend
```

```
demo@mac source .venv/bin/activate  
(.venv) demo@mac
```

```
(.venv) demo@mac pip install torch transformers
```

```
(.venv) demo@mac pip freeze > requirements.txt
```

¹. https://huggingface.co/docs/transformers/main_classes/pipelines

```
1 cd backend
2 source .venv/bin/activate
3 pip install torch transformers
4 pip freeze > requirements.txt
```

Ensuite, nous allons construire notre endpoint dans le fichier `services.py`.

Juste après nos imports existants, nous allons importer `torch`, `transformers` puis paramétrer la démo.

```
import torch
from transformers import pipeline
```

```
USE_DEMO_SENTIMENT = os.environ.get("USE_DEMO_SENTIMENT", "0") == "1"
_sentiment_pipeline = None # Variable pour lazy loading
```

```
1 import torch
2 from transformers import pipeline
3 USE_DEMO_SENTIMENT = os.environ.get("USE_DEMO_SENTIMENT", "0") == "1"
   _sentiment_pipeline = None # Variable pour lazy loading

def get_sentiment_pipeline():
    """
    Charge le pipeline HuggingFace pour l'analyse de sentiment.
    Utilise PyTorch en backend.
    Chargement unique (singleton).
    """
    global _sentiment_pipeline
    if _sentiment_pipeline is None:
        _sentiment_pipeline = pipeline("sentiment-analysis")
    return _sentiment_pipeline

def analyze_sentiment(text):
    """
    Analyse le sentiment du texte passé en argument.

    Retourne : (label, score)
    | - label est 'positive', 'negative', etc.
    Si USE_DEMO_SENTIMENT = 1, retourne ('neutral', 0.5)
    pour accélérer les tests/démo.
    """
    if USE_DEMO_SENTIMENT:
        return "neutral", 0.5

    pipe = get_sentiment_pipeline()
    result = pipe(text)
    label = result[0]["label"].lower()
    score = result[0]["score"]

    return label, score
```

À la fin de notre fichier, nous allons ajouter la fonction de pipeline (lazy loading) ainsi que la fonction de chargement (singleton) et d'analyse :

```
1 def get_sentiment_pipeline():
2     """
3     Charge le pipeline HuggingFace pour l'analyse de sentiment.
4     Utilise PyTorch en backend.
5     Chargement unique (singleton).
6     """
7     global _sentiment_pipeline
8     if _sentiment_pipeline is None:
9         _sentiment_pipeline = pipeline("sentiment-analysis")
10    return _sentiment_pipeline
11
12 def analyze_sentiment(text):
13     if USE_DEMO_SENTIMENT:
14         return "neutral", 0.5
15
16    pipe = get_sentiment_pipeline()
```

```

17 result = pipe(text)
18 label = result[0]["label"].lower()
19 score = result[0]["score"]
20
21 return label, score

```

Bien indenté, votre fichier services.py devrait être similaire à ceci :

```

import json
import os
import torch
from transformers import pipeline

USE_DEMO_TEXT = os.environ.get("USE_DEMO_TEXT", "0") == "1"
USE_DEMO_SENTIMENT = os.environ.get("USE_DEMO_SENTIMENT", "0") == "1"
_sentiment_pipeline = None # Variable pour lazy loading

DATASET_PATH = os.path.join(os.path.dirname(__file__), "../data/products.json")

def load_products():
    with open(DATASET_PATH, 'r', encoding='utf-8') as f:
        return json.load(f)

PRODUCTS_DATA = load_products()

def find_product_by_name(query: str):
    query = query.lower()
    for product in PRODUCTS_DATA:
        if product['name'].lower() in query:
            return product
    return None

def suggest_wine_for_product(product):
    pairing = {
        "Fromage": "Un vin blanc sec, type Sancerre ou Chablis.",
        "Poisson": "Un vin blanc fruité, type Sauvignon ou Chardonnay.",
        "Charcuterie": "Un vin rouge léger, type Pinot Noir.",
        "Dessert": "Un vin moelleux, type Sauternes"
    }
    suggestion = pairing.get(product["category"], "Un vin adapté à ce plat, à voir selon vos goûts !")
    return f"Pour {product['name']}, je te recommande {suggestion}"

def get_product_info(user_message: str):
    if USE_DEMO_TEXT:
        return "Mode DEMO : Produit détecté -> suggestion de vin fictive."
    product = find_product_by_name(user_message)
    if not product:
        return None

    message_lower = user_message.lower()
    if any(x in message_lower for x in [
        "que contient",
        "description",
        "qu'y a-t-il dans",
        "c'est quoi",
        "qu'est-ce que"
    ]):
        return f"{product['name']} : {product['description']}"
    else:
        return suggest_wine_for_product(product)

# Analyse de sentiment
def get_sentiment_pipeline():
    """
    Charge le pipeline HuggingFace pour l'analyse de sentiment.
    Utilise PyTorch en backend.
    Chargement unique (singleton).
    """
    global _sentiment_pipeline
    if _sentiment_pipeline is None:
        _sentiment_pipeline = pipeline("sentiment-analysis")
    return _sentiment_pipeline

def analyze_sentiment(text):
    if USE_DEMO_SENTIMENT:
        return "neutral", 0.5

    pipe = get_sentiment_pipeline()
    result = pipe(text)
    label = result[0]["label"].lower()
    score = result[0]["score"]

    return label, score

```

Ensuite, dans notre fichier main.py, nous allons améliorer notre endpoint chat en intégrant l'analyse de sentiment juste après la génération de la recommandation.

Puis nous allons retourner la réponse.

```
@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)

    if dataset_response:
        recommendation = dataset_response
    else:
        prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en français. Que recommandes-tu comme vin ?"
        recommendation = await run_in_threadpool(llm_client.complete, prompt)

    # --- Analyse de sentiment avancée ---
    sentiment, score = analyze_sentiment(request.user_message)
    escalate = (sentiment == "negative" and score > 0.75)

    # Affichage côté terminal pour debug :
    if escalate:
        print(f"[ESCALADE] Message critique détecté : \"{request.user_message}\" (sentiment : {sentiment}, score : {score})")
    else:
        print(f"[INFO] Message traité : \"{request.user_message}\" (sentiment : {sentiment}, score : {score})")

    return ChatResponse(
        recommendation=recommendation,
        sentiment=sentiment,
        escalate_to_human=escalate
    )
```

```
1 @app.post("/chat", response_model=ChatResponse)
2 async def chat_endpoint(request: ChatRequest):
3     dataset_response = get_product_info(request.user_message)
4     if dataset_response:
5         recommendation = dataset_response
6     else:
7         prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en
            français. Que recommandes-tu comme vin ?"
8         recommendation = await run_in_threadpool(llm_client.complete, prompt)
9
10 # --- Analyse de sentiment avancée ---
11 sentiment, score = analyze_sentiment(request.user_message)
12 escalate = (sentiment == "negative" and score > 0.75)
13
14 # Affichage côté terminal pour debug :
15 if escalate:
16     print(f"[ESCALADE] Message critique détecté : \"{request.user_message}\" (sentiment :
        {sentiment}, score : {score})")
17 else:
18     print(f"[INFO] Message traité : \"{request.user_message}\" (sentiment : {sentiment},
        score : {score})")
19
20 return ChatResponse(
21     recommendation=recommendation,
22     sentiment=sentiment,
23     escalate_to_human=escalate
24 )
25 )
```


Enfin, nous allons importer notre fonction dans notre fichier main.py

```
from core.services import analyze_sentiment
```

```
1 from core.services import analyze_sentiment
```

Bien indenté, votre fichier main.py devrait être similaire à ceci.

```

from fastapi import FastAPI
from api.schemas import ChatRequest, ChatResponse
from core.services import get_product_info
from core.llm_client import LLMClient
from starlette.concurrency import run_in_threadpool
from fastapi.middleware.cors import CORSMiddleware
from core.services import analyze_sentiment
from fastapi import FastAPI, UploadFile, File
from core.vision_client import analyze_image

app = FastAPI()
llm_client = LLMClient()

origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173"
]

# Endpoint Chat

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)

    if dataset_response:
        recommendation = dataset_response
    else:
        prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en français. Que recommandes-tu comme vin ?"
        recommendation = await run_in_threadpool(llm_client.complete, prompt)

    # --- Analyse de sentiment avancée ---
    sentiment, score = analyze_sentiment(request.user_message)
    escalate = (sentiment == "negative" and score > 0.75)

    # Affichage côté terminal pour debug :
    if escalate:
        print(f"[ESCALADE] Message critique détecté : \"{request.user_message}\" (sentiment : {sentiment}, score : {score})")
    else:
        print(f"[INFO] Message traité : \"{request.user_message}\" (sentiment : {sentiment}, score : {score})")

    return ChatResponse(
        recommendation=recommendation,
        sentiment=sentiment,
        escalate_to_human=escalate
    )

# Endpoint Image

@app.post("/chat/photo")
async def chat_photo(file: UploadFile = File(...)):
    # 1. Enregistre l'image temporairement sur le disque
    path = f"/tmp/{file.filename}"
    contents = await file.read()
    with open(path, "wb") as f:
        f.write(contents)

    # 2. Analyse l'image
    desc = analyze_image(path)

    return {"description": desc}

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

1 from fastapi import FastAPI
2 from api.schemas import ChatRequest, ChatResponse
3 from core.services import get_product_info
4 from core.llm_client import LLMClient
5 from starlette.concurrency import run_in_threadpool
6 from fastapi.middleware.cors import CORSMiddleware
7 from core.services import analyze_sentiment
8
9 app = FastAPI()
10 llm_client = LLMClient()
11
12 origins = [
13 "http://localhost:5173",
14 "http://127.0.0.1:5173"
15 ]

```

```

16
17 # Endpoint Chat
18
19 @app.post("/chat", response_model=ChatResponse)
20 async def chat_endpoint(request: ChatRequest):
21     dataset_response = get_product_info(request.user_message)
22
23     if dataset_response:
24         recommendation = dataset_response
25     else:
26         prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en
                français. Que recommandes-tu comme vin ?"
27         recommendation = await run_in_threadpool(llm_client.complete, prompt)
28
29     # --- Analyse de sentiment avancée ---
30     sentiment, score = analyze_sentiment(request.user_message)
31     escalate = (sentiment == "negative" and score > 0.75)
32
33     # Affichage côté terminal pour debug :
34     if escalate:
35         print(f"[ESCALADE] Message critique détecté : \"{request.user_message}\" (sentiment :
                {sentiment}, score : {score})")
36     else:
37         print(f"[INFO] Message traité : \"{request.user_message}\" (sentiment : {sentiment},
                score : {score})")
38
39
40     return ChatResponse(
41         recommendation=recommendation,
42         sentiment=sentiment,
43         escalate_to_human=escalate
44     )
45
46 # Endpoint Image
47
48 @app.post("/chat/photo")
49 async def chat_photo(file: UploadFile = File(...)):
50     # 1. Enregistre l'image temporairement sur le disque
51     path = f"/tmp/{file.filename}"
52     contents = await file.read()
53     with open(path, "wb") as f:
54         f.write(contents)
55
56     # 2. Analyse l'image
57     desc = analyze_image(path)
58
59     return {"description": desc}
60
61
62
63 app.add_middleware(
64     CORSMiddleware,
65     allow_origins=origins,
66     allow_credentials=True,
67     allow_methods=["*"],
68     allow_headers=["*"],
69 )

```

Testons notre nouveau endpoint depuis notre backend. Le venv activé, activez la démo pour la fonction text et sentiment.

```
(.venv) demo@mac USE_DEMO_TEXT=0 USE_DEMO_SENTIMENT=0 uvicorn api.main:app --reload
```

```
1 export USE_DEMO_TEXT=0
2 export USE_DEMO_SENTIMENT=0
3 uvicorn api.main:app --reload
```

```
demo@dev npm run dev
```

Puis lancez votre serveur frontend : "npm run dev."

Depuis la fonction chat inscrivez un message négatif qui ne laisse pas place à l'interprétation.

Ensuite, rendez-vous dans votre terminal et observez la réponse JSON. Le champ sentiment doit indiquer "negative" et le champ escalate_to_human doit passer à true.

La réponse attendue est la suivante côté backend.

```
Device set to use mps:0  
[ESCALADE] Message critique détecté : "Ce service est inacceptable !" (sentiment : negative, score :  
0.9989192485809326)  
INFO: 127.0.0.1:49900 - "POST /chat HTTP/1.1" 200 OK
```

Conciergerie Bon Vivant

A **Augustin** 

Bon Vivant - Votre conciergerie intelligente

Bienvenue, Augustin pour vous servir. Vous hésitez sur un vin, cherchez le fromage parfait ou rêvez d'une association inédite? Confiez-moi vos attentes, je me charge du reste ! 

Ce service est inacceptable !

. Je suis désolé d'apprendre que vous ne vous sentez pas satisfait avec notre sélection de vins. Je comprends que les goûts et les préférences sont personnels et que chacun cherche quelque chose différent dans un vin.



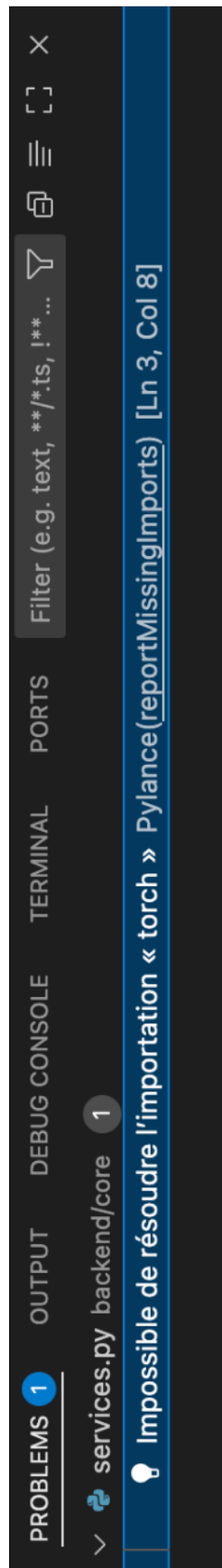
Comment puis-je vous aider ?



On voit également une réponse adaptée côté front. On pourrait améliorer la réponse du chat via du prompting, mais on voit déjà que le chatbot a compris l'humeur de l'utilisateur.

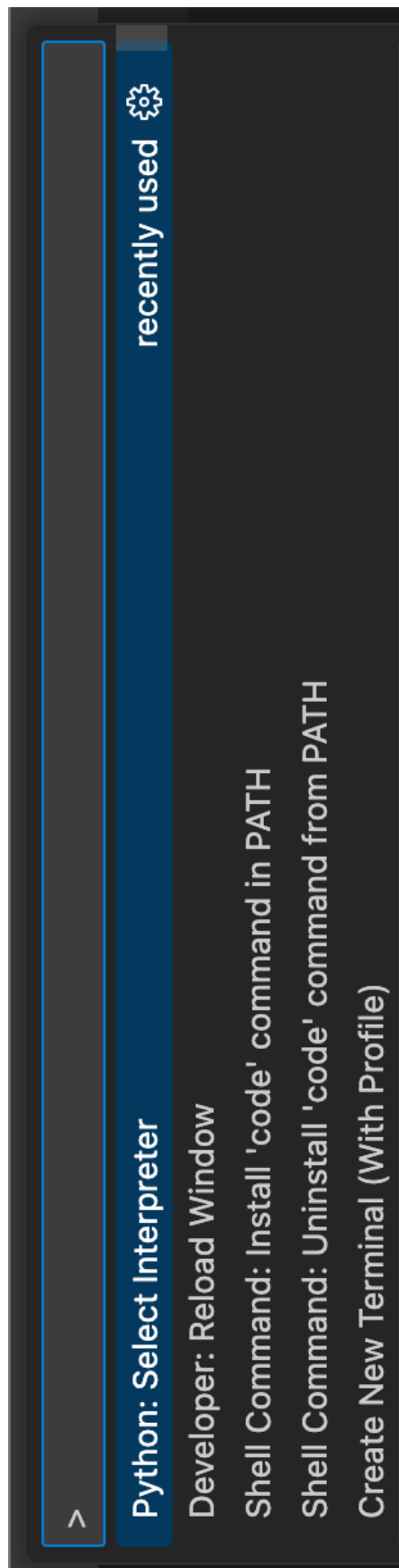
 Remarque

Il se peut que vous ayez une erreur dans votre terminal et que cela vous bloque côté serveur.



Dans la plupart des cas, cela est dû au fait que votre terminal n'utilise pas le venv python.
Sur VS code, il vous suffit de vous rendre dans la palette (commande + shift + P)

Cela ouvrira une fenêtre :

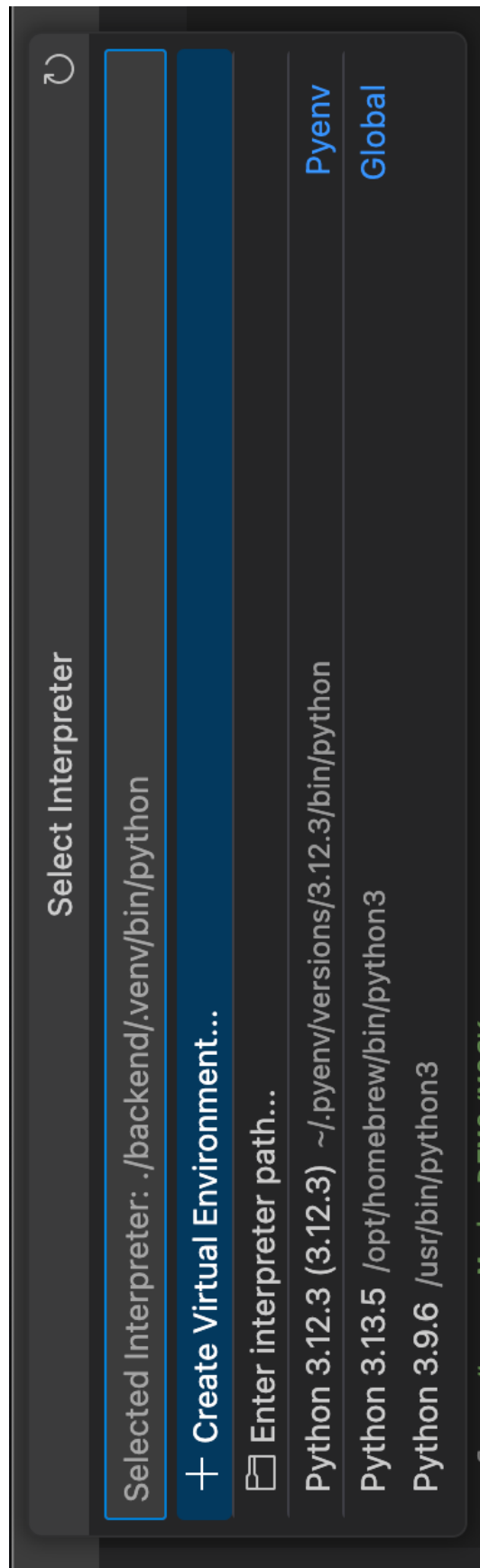


Ensuite, tapez “Python Select Interpreter”

Rendez-vous dans votre terminal, dossier backend puis activez votre venv.

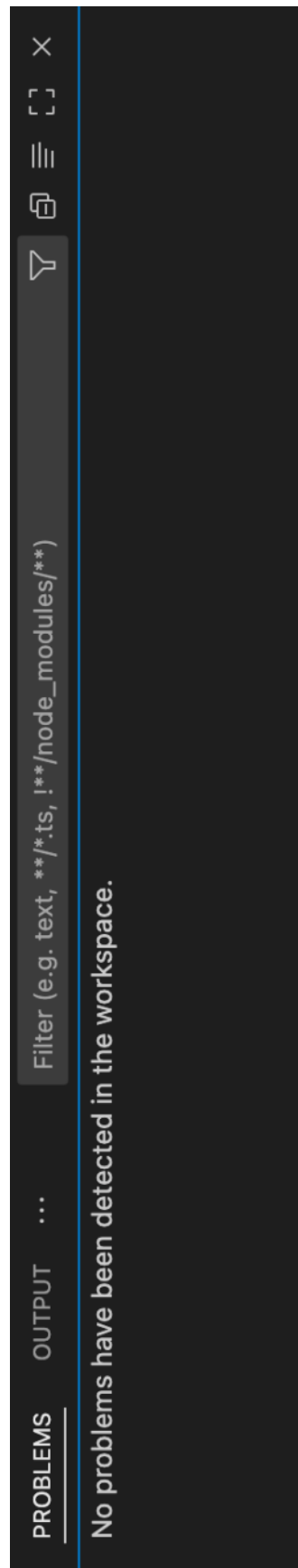
Tapez ensuite la commande `pwd` pour connaître le chemin exact de votre venv.

Copiez puis collez le résultat dans le commande panel.



Quittez puis relancez VS code.

L'erreur devrait avoir disparue.



II Chatbot : ajout endpoint "Image"

De nos jours, l'expérience client ne passe plus seulement par le texte. Les utilisateurs veulent pouvoir poser des questions, envoyer une photo d'un produit ou même utiliser leur voix pour interagir avec une entreprise.

C'est l'essence même d'une IA multimodale. Il s'agit d'une Intelligence artificielle capable de comprendre plusieurs types d'entrées afin d'offrir un service plus riche et interactif.

Pour cette première version, nous allons implémenter un endpoint qui permettra à notre chatbot d'analyser une image. Pour ce faire, nous allons utiliser le modèle Llama3.3-vision via Ollama.

Cela permettra au chatbot de décrire de manière détaillée une photo donnée.

Ainsi, pensez bien à télécharger ollama sur votre machine ainsi que le modèle Llama3.2-vision avant de passer à la suite. Lancez ces commandes suivantes en dehors de votre projet, dans un autre terminal.

```
demo@mac brew install ollama
```

```
demo@mac ollama serve
```

```
demo@mac ollama pull llama3.2-vision:latest
```

```
demo@mac ollama list
NAME                                ID                                SIZE  MODIFIED
llama3.2-vision:latest             6f2f9757ae97                   7.8 GB 4 weeks ago
demo@mac
```

```
1 Mac os
2 brew install ollama
3 ollama serve
4 Linux
5 curl -fsSL https://ollama.com/install.sh | sh
```

Sous windows : Download Ollama²

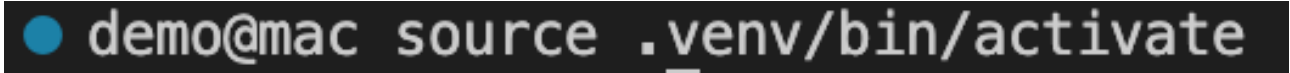
Ensuite téléchargez le modèle Vision.

```
1 ollama pull llama3.2-vision:latest
2 ollama list
```

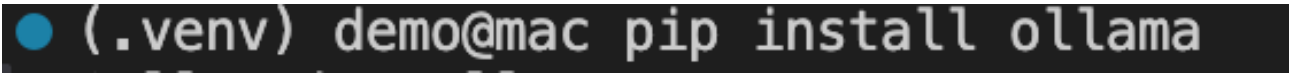
De retour dans notre projet, nous pouvons allons installer le package Python ollama, ainsi que la bibliothèque Python Multipart. Cette dernière nous permettra d'accepter des fichiers. Dans notre cas présent, une image. Pensez bien à vous rendre dans votre backend, puis activez votre environnement virtuel.



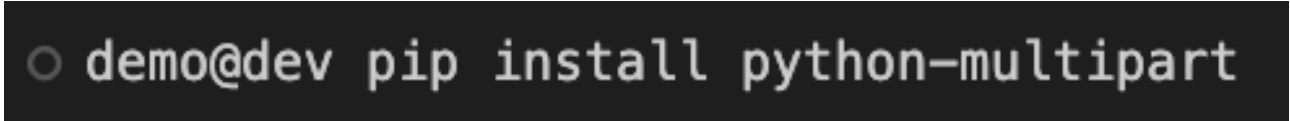
```
● demo@mac cd backend
```



```
● demo@mac source .venv/bin/activate
```



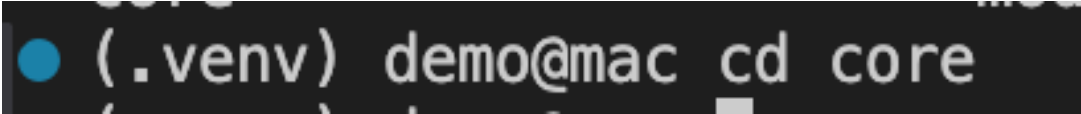
```
● (.venv) demo@mac pip install ollama
```



```
○ demo@dev pip install python-multipart
```

```
1 cd backend
2 source .venv/bin/activate
3 pip install ollama
4 pip install python-multipart
```

La première étape consiste à créer un fichier dans lequel nous stockerons toute la logique métier liée à l'image.



```
● (.venv) demo@mac cd core
```

² <https://ollama.com/download>

```
1 cd core
2 touch vision_client.py
```

```
● (.venv) demo@mac touch vision_client.py
```

Ensuite, nous allons coder notre endpoint dans le fichier main.py

Nous commencerons par enregistrer temporairement l'image sur le disque

```
@app.post("/chat/photo")
async def chat_photo(file: UploadFile = File(...)):
```

```
1 async def chat_photo(file: UploadFile = File(...)):
```

```
    path = f"/tmp/{file.filename}"
```

Ensuite, nous allons décrire le chemin où nous allons sauvegarder l'image

```
1 path = f"/tmp/{file.filename}"
```

Enfin, nous allons lire le contenu fichier uploadé puis ouvrir/créer le fichier

```
    contents = await file.read()
    with open(path, "wb") as f:
        f.write(contents)
```

```
1 contents = await file.read()
2     with open(path, "wb") as f:
3         f.write(contents)
```

Cette première étape étant faite nous allons nous attaquer à l'analyse de l'image.

Pour ce faire, nous allons passer le chemin de l'image à la fonction d'analyse IA puis renvoyer la description produite par l'IA au frontend

```
    desc = analyze_image(path)
    return {"description": desc}
```

```
1 desc = analyze_image(path)
2     return {"description": desc}
```



```
@app.post("/chat/photo")
async def chat_photo(file: UploadFile = File(...)):
    # 1. Enregistre l'image temporairement sur le disque
    path = f"/tmp/{file.filename}"
    contents = await file.read()
    with open(path, "wb") as f:
        f.write(contents)

    # 2. Analyse l'image
    desc = analyze_image(path)
    return {"description": desc}
```

Le bloc de code complet, bien indenté devrait ressembler à ceci.

```
1 from fastapi import FastAPI, UploadFile, File from core.vision_client import
  analyze_image
2 app = FastAPI()
3 @app.post("/chat/photo") async def chat_photo(file: UploadFile = File(...)): # 1.
  Enregistre l'image temporairement sur le disque path = f"/tmp/{file.filename}"
  contents = await file.read() with open(path, "wb") as f: f.write(contents)
4
5 desc = analyze_image(path)
6
7
8 return {"description": desc}
```

Et le fichier main.py au total devrait être similaire à ceci.

```
from fastapi import FastAPI
from api.schemas import ChatRequest, ChatResponse
from core.services import get_product_info
from core.llm_client import LLMClient
from starlette.concurrency import run_in_threadpool
from fastapi.middleware.cors import CORSMiddleware
from core.services import analyze_sentiment
from fastapi import FastAPI, UploadFile, File
from core.vision_client import analyze_image

app = FastAPI()
llm_client = LLMClient()

origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173"
]

# Endpoint Chat

@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)

    if dataset_response:
        recommendation = dataset_response
    else:
        prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en français. Que recommandes-tu comme vin ?"
        recommendation = await run_in_threadpool(llm_client.complete, prompt)

    # --- Analyse de sentiment avancée ---
    sentiment, score = analyze_sentiment(request.user_message)
    escalate = (sentiment == "negative" and score > 0.75)

    # Affichage côté terminal pour debug :
    if escalate:
        print(f"[ESCALADE] Message critique détecté : \"{request.user_message}\" (sentiment : {sentiment}, score : {score})")
    else:
        print(f"[INFO] Message traité : \"{request.user_message}\" (sentiment : {sentiment}, score : {score})")

    return ChatResponse(
        recommendation=recommendation,
        sentiment=sentiment,
        escalate_to_human=escalate
    )

# Endpoint Image

@app.post("/chat/photo")
async def chat_photo(file: UploadFile = File(...)):
    # 1. Enregistre l'image temporairement sur le disque
    path = f"/tmp/{file.filename}"
    contents = await file.read()
    with open(path, "wb") as f:
        f.write(contents)

    # 2. Analyse l'image
    desc = analyze_image(path)

    return {"description": desc}

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)
```

```
1 from fastapi import FastAPI
2 from api.schemas import ChatRequest, ChatResponse
3 from core.services import get_product_info
4 from core.llm_client import LLMClient
5 from starlette.concurrency import run_in_threadpool
6 from fastapi.middleware.cors import CORSMiddleware
7 from core.services import analyze_sentiment
8 from fastapi import FastAPI, UploadFile, File
9 from core.vision_client import analyze_image
10
11 app = FastAPI()
12 llm_client = LLMClient()
13
14 origins = [
15 "http://localhost:5173",
```

```

16 "http://127.0.0.1:5173"
17 ]
18
19 # Endpoint Chat
20
21 @app.post("/chat", response_model=ChatResponse)
22 async def chat_endpoint(request: ChatRequest):
23     dataset_response = get_product_info(request.user_message)
24
25     if dataset_response:
26         recommendation = dataset_response
27     else:
28         prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en
                français. Que recommandes-tu comme vin ?"
29         recommendation = await run_in_threadpool(llm_client.complete, prompt)
30
31     # --- Analyse de sentiment avancée ---
32     sentiment, score = analyze_sentiment(request.user_message)
33     escalate = (sentiment == "negative" and score > 0.75)
34
35     # Affichage côté terminal pour debug :
36     if escalate:
37         print(f"[ESCALADE] Message critique détecté : \"{request.user_message}\" (sentiment :
                {sentiment}, score : {score})")
38     else:
39         print(f"[INFO] Message traité : \"{request.user_message}\" (sentiment : {sentiment},
                score : {score})")
40
41
42     return ChatResponse(
43         recommendation=recommendation,
44         sentiment=sentiment,
45         escalate_to_human=escalate
46     )
47
48 # Endpoint Image
49
50 @app.post("/chat/photo")
51 async def chat_photo(file: UploadFile = File(...)):
52     # 1. Enregistre l'image temporairement sur le disque
53     path = f"/tmp/{file.filename}"
54     contents = await file.read()
55     with open(path, "wb") as f:
56         f.write(contents)
57
58     # 2. Analyse l'image
59     desc = analyze_image(path)
60     return {"description": desc}
61
62
63
64 app.add_middleware(
65     CORSMiddleware,
66     allow_origins=origins,
67     allow_credentials=True,
68     allow_methods=["*"],
69     allow_headers=["*"],
70 )

```

Ceci étant fait, nous pouvons modifier notre fichier vision_client.py

Dans ce fichier nous allons créer notre fonction `analyze_image` que nous avons appelé dans notre endpoint.

Nous commençons par nos imports, puis nous créons notre mode démo.

```
import os  
import ollama
```

```
USE_DEMO_IMAGE = os.environ.get("USE_DEMO_IMAGE", "0") == "1"
```

```
1 import os  
2 import ollama  
3 USE_DEMO_IMAGE = os.environ.get("USE_DEMO_IMAGE", "0") == "1"
```

Ensuite, nous allons créer notre fonction d'analyse.

Celle-ci va retourner une description soit via le modèle IA soit en mode démo.

Nous passons à l'argument le chemin absolu de l'image à analyser retournons la description réelle ou simulée en mode démo pour ne pas bloquer les scripts comme vu précédemment.

```
def analyze_image(path: str) -> str:
    if USE_DEMO_IMAGE:
        return (
            "Ceci est une description simulée (mode démo activé) "
            "Aucune analyse réelle de l'image n'a été effectuée."
        )
```

```
1 def analyze_image(path: str) -> str:
2
3 if USE_DEMO_IMAGE:
4 return (
5 "Ceci est une description simulée (mode démo activé) "
6 "Aucune analyse réelle de l'image n'a été effectuée."
7 )
```

Dans le cas où ce ne serait pas une démo, nous appelons réellement le modèle IA.

```
resp = ollama.chat(  
    model="llama3.2-vision:latest",  
    messages=[  
        {  
            "role": "user",  
            "content": "Décris cette image en détail.",  
            "images": [path]  
        }  
    ]  
)  
return resp["message"]["content"].strip()
```

```
1 resp = ollama.chat(  
2     model="llama3.2-vision:latest",  
3     messages=[  
4         "role": "user",  
5         "content": "Décris cette image en détail.",  
6         "images": [path]  
7     ]  
8 )  
9 return resp["message"]["content"].strip()
```

Votre fichier vision_client.py au complet doit être similaire à celui-ci.

```
import os
import ollama

USE_DEMO_IMAGE = os.environ.get("USE_DEMO_IMAGE", "0") == "1"

def analyze_image(path: str) -> str:
    if USE_DEMO_IMAGE:
        return (
            "Ceci est une description simulée (mode démo activé) "
            "Aucune analyse réelle de l'image n'a été effectuée."
        )
    # Sinon, appel réel au modèle IA
    resp = ollama.chat(
        model="llama3.2-vision:latest",
        messages=[
            {
                "role": "user",
                "content": "Décris cette image en détail.",
                "images": [path]
            }
        ]
    )
    return resp["message"]["content"].strip()
```

```
1 import os
2 import ollama
3
4 USE_DEMO_IMAGE = os.environ.get("USE_DEMO_IMAGE", "0") == "1"
5
6
7 def analyze_image(path: str) -> str:
8     """
9     Analyse une image et retourne une description, soit via le modèle IA, soit en mode
10    demo.
11
12    Args:
13    path (str): Chemin absolu de l'image à analyser.
14
15    Returns:
16    str: Description générée (réelle ou simulée en mode demo)
17    """
18    if USE_DEMO_IMAGE:
19        return (
20            "Ceci est une description simulée (mode démo activé) "
```

```
19 "Aucune analyse réelle de l'image n'a été effectuée."  
20 )  
21  
22 # Sinon, appel réel au modèle IA  
23 resp = ollama.chat(  
24 model="llama3.2-vision:latest",  
25 messages=[  
26 "role": "user",  
27 "content": "Décris cette image en détail.",  
28 "images": [path]  
29 ]]  
30 )  
31 return resp["message"]["content"].strip()
```

Il est maintenant temps de tester notre endpoint. Rendez-vous dans votre dossier backend puis lancez votre serveur uvicorn en paramétrant correctement les variables de démo.

```
○ demo@dev cd ..
```

```
○ demo@dev USE_DEMO_TEXT=1 USE_DEMO_SENTIMENT=1 USE_DEMO_IMAGE=0 uvicorn api.main:app --reload
```



```
1 cd ..  
2 USE_DEMO_TEXT=1 USE_DEMO_SENTIMENT=1 USE_DEMO_IMAGE=0 uvicorn api.main:app --reload
```

Ensuite lancez votre serveur frontend dans un autre terminal.

```
demo@dev npm run dev
```

```
1 npm run dev
```

Pour ce faire, placez-vous dans le dossier tests de votre projet.

Téléchargez une photo sur internet en veillant à ce que le format de l'image soit en JPEG (.jpg ou jpeg) car il s'agit du format connu par tous les modèles IA.

Nous vous conseillons également une taille d'image peu élevée (maximum 600 x 500 px). En effet, nous souhaitons éviter le testing avec les images lourdes car le traitement par l'IA risque d'être trop long.

À titre indicatif, l'image utilisée de notre côté pour ce test est de 88 ko.



Source : Pixabay³

Chat

Newsletter

Upsell

Conciergerie Bon Vivant 🛎

A Augustin 🎩

Bon Vivant - Votre conciergerie intelligente

(Photo envoyée)

Here is a concise summary of the image: The image features a wooden cutting board or table laden with an assortment of cheeses, accompanied by fresh fruit and nuts. The cheeses are presented in various shapes and sizes, with some appearing to be soft-ripened, while others have a harder, more aged texture. The cheeses are arranged in a visually appealing manner, with some placed on their sides and others sliced into wedges. The cheeses are accompanied by a selection of fresh fruit, including grapes, strawberries, and figs, as well as a few nuts, such as walnuts and almonds. The overall presentation suggests that the image is intended to showcase a cheese platter or board, perhaps for a



Comment puis-je vous aider ?

**Remarque**

Par défaut, le LLM répond dans sa langue native, à savoir l'anglais mais il est possible de forcer l'utilisation du français. Toutefois, pour notre V1, cela permet de démontrer que notre endpoint fonctionne correctement.

³. https://cdn.pixabay.com/photo/2015/12/26/11/08/kaseplatte-1108564_1280.jpg

III Chatbot : ajout endpoint "Voix"

Désormais, nous allons enrichir l'expérience utilisateur du chatbot en permettant aux clients de poser leurs questions oralement.

👁 Exemple

Quel vin choisir avec mon plateau de fromage ?

L'objectif ? Accessibilité, rapidité et modernité.

Pour créer notre endpoint voix, nous allons suivre le même processus que précédemment. Nous allons commencer par importer nos dépendances, télécharger notre modèle IA, puis créer la logique de notre endpoint.

Concernant les dépendances, nous les avons téléchargés en début de projet (pip install fastapi uvicorn vosk pytsx3...)

La seconde étape consiste donc à télécharger notre modèle Vosk.

Vulgarisation concepts IA et définition de Vosk

Az Définition

Parmi les termes à connaître à cette étape il y a :

Premièrement TTS pour Text-to-Speech. TTS c'est le fait de transformer une réponse texte en voix synthétique pour un retour audio. C'est la fonction de pytsx3.

Ensuite, il y a le STT pour Speech-to-text. C'est le fait de transformer la voix en texte. Ici, ce sera le rôle de Vosk.

D'ailleurs qu'est-ce que Vosk ?

Vosk est un modèle de reconnaissance vocale. Il en existe plusieurs modèles et pour notre exemple, nous choisirons le modèle anglais car il est léger et performant.

💬 Remarque

Si vous êtes sous Mac OS, pensez à télécharger wget car il n'est pas installé par défaut sur mac et nous en aurons besoin pour télécharger notre modèle Vosk.

Dans votre terminal, et depuis n'importe quel dossier (par exemple, à la racine de votre mac), tapez la commande suivante.

```
[demo@mac brew install wget
```

Endpoint Voix Méthode

```
demo@dev cd backend
```

```
demo@dev cd data
```

```
demo@dev
```

```
demo@dev wget https://alphacephei.com/vosk/models/vosk-model-small-en-us-0.15.zip
```

```
demo@dev unzip vosk-model-small-en-us-0.15.zip
```

```
demo@dev rm vosk-model-small-en-us-0.15.zip
```

```
1 cd backend
2 cd data
3 wget https://alphacephei.com/vosk/models/vosk-model-small-en-us-0.15.zip
4 unzip vosk-model-small-en-us-0.15.zip
5 rm vosk-model-small-en-us-0.15.zip
```

 Attention

Nous venons de télécharger le modèle VOSK. Pensez à l'inclure dans le fichier .gitignore avant de poursuivre.

```

.gitignore
1 .env/
2 backend/.env/
3 node_modules/
4 frontend/node_modules/
5
6 data/raw/
7 data/processed/
8 backend/models/
9
10 *.gguf
11 *.bin
12 *.ckpt
13
14 mlruns/
15 __pycache__/
16 .DS_Store
17 backend/data/vosk-model-small-en-us-0.15/
18

```

```
1 backend/data/vosk-model-small-en-us-0.15/
```

Nous allons commencer par créer notre fichier voice_service.py. Dans le dossier core.

Comme pour l'endpoint image, ce fichier centralise toutes les fonctions de traitement de voix.

```
demo@dev cd ..
```

```
demo@dev cd backend
```

```
demo@dev cd core
```

```
demo@dev touch voice_service.py
```

```
1 cd ..  
2 cd backend  
3 cd core  
4 touch voice_service.py
```

Ensuite, dans ce même fichier nous allons commencer par importer nos dépendances. Vosk, pour la reconnaissance de voix, pyttsx3 pour la synthèse vocale, json pour les résultats Vosk ainsi que wave pour lire les fichiers audio .wav

Nous allons également importer os pour notre variable d'environnement.

```
from vosk import Model, KaldiRecognizer  
import pyttsx3  
import json  
import wave  
import os
```

```
1 from vosk import Model, KaldiRecognizer  
2 import pyttsx3  
3 import json  
4 import wave  
5 import os
```

```
# Charger le modèle Vosk
model = Model("data/vosk-model-small-en-us-0.15")
# Initialiser le moteur de synthèse vocale
engine = pyttsx3.init()
```

Par la suite, nous allons initialiser les modèles de sorte que VOSK charge une seule fois.

Nous allons ensuite initialiser le moteur de synthèse vocale pyttsx3

```
1 model = Model("data/vosk-model-small-en-us-0.15")
2 engine = pyttsx3.init()
```



```
def transcribe_audio(wav_path):  
    wf = wave.open(wav_path, "rb")  
    rec = KaldiRecognizer(model, wf.getframerate())  
    text = ""  
  
    while True:  
        data = wf.readframes(4000)  
        if not data:  
            break  
        if rec.AcceptWaveform(data):  
            res = json.loads(rec.Result())  
            text += " " + res.get("text", "")  
  
    res = json.loads(rec.FinalResult())  
    return (text + " " + res.get("text", "")).strip()
```

Ensuite nous allons créer une fonction qui va commencer par ouvrir le fichier audio .wav, créer un objet avec le modèle ainsi que la fréquence d'échantillonnage du fichier audio.

Il va lire l'audio en blocs (ici 4 000 frames à la fois pour plus d'efficacité).

Pour chaque bloc, nous allons vérifier si Vosk accepte et reconnaît une phrase complète.

Si oui, celui-ci extrait le texte et l'ajoute à la transcription globale.

Vient ensuite la récupération des derniers mots reconnus, la concaténation et il retourne enfin la transcription complète du fichier audio.

```
1 def transcribe_audio(wav_path):  
2     wf = wave.open(wav_path, "rb")  
3     rec = KaldiRecognizer(model, wf.getframerate())  
4     text = ""  
5  
6     while True:  
7         data = wf.readframes(4000)  
8         if not data:  
9             break
```

```
10 if rec.AcceptWaveform(data):  
11     res = json.loads(rec.Result())  
12     text += " " + res.get("text", "")  
13  
14 res = json.loads(rec.FinalResult())  
15 return (text + " " + res.get("text", "")).strip()
```

Enfin, nous finissons par la fonction de synthèse vocale. Nous allons utiliser pyttsx3 pour transformer le texte en fichier audio.

```
def speak_text(text, out_path):  
    engine.save_to_file(text, out_path)  
    engine.runAndWait()
```

```
1 def speak_text(text, out_path):  
2     engine.save_to_file(text, out_path)  
3     engine.runAndWait()
```

Votre fichier voice_service.py au complet devrait être similaire a ceci.

```
from vosk import Model, KaldiRecognizer
import pytttsx3
import json
import wave
import os

USE_DEMO_VOICE = os.environ.get("USE_DEMO_VOICE", "0") == "1"

# Charger le modèle Vosk
model = Model("data/vosk-model-small-en-us-0.15")
# Initialiser le moteur de synthèse vocale
engine = pytttsx3.init()

def transcribe_audio(wav_path):
    wf = wave.open(wav_path, "rb")
    rec = KaldiRecognizer(model, wf.getframerate())
    text = ""

    while True:
        data = wf.readframes(4000)
        if not data:
            break
        if rec.AcceptWaveform(data):
            res = json.loads(rec.Result())
            text += " " + res.get("text", "")

    res = json.loads(rec.FinalResult())
    return (text + " " + res.get("text", "")).strip()

def speak_text(text, out_path):
    engine.save_to_file(text, out_path)
    engine.runAndWait()
```

```
1 from vosk import Model, KaldiRecognizer
2 import pytttsx3
3 import json
4 import wave
5 import os
6
7 USE_DEMO_VOICE = os.environ.get("USE_DEMO_VOICE", "0") == "1"
8
9 # Charger le modèle Vosk
10 model = Model("data/vosk-model-small-en-us-0.15")
11 # Initialiser le moteur de synthèse vocale
12 engine = pytttsx3.init()
13
14
15 def transcribe_audio(wav_path):
16 wf = wave.open(wav_path, "rb")
```

```
17 rec = KaldiRecognizer(model, wf.getframerate())
18 text = ""
19
20 while True:
21 data = wf.readframes(4000)
22 if not data:
23 break
24 if rec.AcceptWaveform(data):
25 res = json.loads(rec.Result())
26 text += " " + res.get("text", "")
27
28 res = json.loads(rec.FinalResult())
29 return (text + " " + res.get("text", "")).strip()
30
31
32 def speak_text(text, out_path):
33 engine.save_to_file(text, out_path)
34 engine.runAndWait()
```

Ensuite, passons au fichier main.py.

Dans le fichier main.py nous allons construire la structure de notre endpoint voice chat.

Nous commençons comme précédemment par nos imports en début de fichier.

```
from core.voice_service import USE_DEMO_VOICE, transcribe_audio, speak_text
from fastapi import BackgroundTasks
import tempfile
import os
```

```
1 from core.voice_service import USE_DEMO_VOICE, transcribe_audio, speak_text
2 from fastapi import BackgroundTasks
3 import tempfile
4 import os
```

Puis nous allons créer notre condition démo qui simulera un fichier audio si la démo est activée.

Ceci afin d'éviter que le modèle Vosk soit chargé lors du testing des autres endpoints.

```
@app.post("/voice-chat")
async def voice_chat(
    audio: UploadFile = File(...),
    background_tasks: BackgroundTasks = None
):
    if USE_DEMO_VOICE:
        # --- Mode démo rapide ---
        demo_transcript = "Bonjour, ceci est une transcription simulée."
        demo_ai_answer = "Voici une réponse IA simulée par le mode démo."
        demo_audio_filename = "demo.wav"

        # Simule un fichier audio demo
        demo_path = os.path.join(tempfile.gettempdir(), demo_audio_filename)
        with open(demo_path, "wb") as f:
            f.write(b"\x00\x00") # Écrit un mini fichier wav vide (ça évite l'erreur 404)

        return {
            "transcript": demo_transcript,
            "ai_answer": demo_ai_answer,
            "audio_filename": demo_audio_filename
        }
```

```
1 @app.post("/voice-chat")
2 async def voice_chat(
3 audio: UploadFile = File(...),
4 background_tasks: BackgroundTasks = None
5 ):
6
7 if USE_DEMO_VOICE:
8 # --- Mode démo rapide ---
9 demo_transcript = "Bonjour, ceci est une transcription simulée."
10 demo_ai_answer = "Voici une réponse IA simulée par le mode démo."
11 demo_audio_filename = "demo.wav"
12
13 # Simule un fichier audio demo
14 demo_path = os.path.join(tempfile.gettempdir(), demo_audio_filename)
15 with open(demo_path, "wb") as f:
16 f.write(b"\x00\x00") # Écrit un mini fichier wav vide (ça évite l'erreur 404)
```

```
17
18 return {
19 "transcript": demo_transcript,
20 "ai_answer": demo_ai_answer,
21 "audio_filename": demo_audio_filename
22 }
```

Ensuite, nous allons créer la pipeline réelle.

Nous commençons par enregistrer l'audio temporairement.

```
# --- Pipeline réelle ---
# 1. Enregistre l'audio temporaire
temp_in = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
content = await audio.read()
temp_in.write(content)
temp_in.flush()
temp_in.close()
```

```
1 temp_in = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
2 content = await audio.read()
3 temp_in.write(content)
4 temp_in.flush()
5 temp_in.close()
```

Ensuite nous allons transcrire l'audio en texte.

```
# 2. Transcrit l'audio en texte  
transcribed_text = transcribe_audio(temp_in.name)
```

```
1 transcribed_text = transcribe_audio(temp_in.name)
```


Nous passons le texte transcrit précédemment à l'IA pour obtenir une recommandation.

```
# 3. Passe ce texte à l'IA pour obtenir la réponse IA
prompt = f"L'utilisateur dit : {transcribed_text}. Que recommandes-tu comme vin ?"
ai_response = await run_in_threadpool(llm_client.complete, prompt)
```

```
1 prompt = f"L'utilisateur dit : {transcribed_text}. Que recommandes-tu comme vin ?"
  ai_response = await run_in_threadpool(llm_client.complete, prompt)
```

Nous synthétisons ensuite la réponse en audio puis nous nettoyons le fichier avant de retourner la réponse

```
# 5. Nettoyage
if background_tasks is not None:
    background_tasks.add_task(os.remove, temp_in.name)
    background_tasks.add_task(os.remove, temp_out.name)

filename = os.path.basename(temp_out.name)

return {
    "transcript": transcribed_text,
    "ai_answer": ai_response,
    "audio_filename": filename
}
```

```
1 if background_tasks is not None: background_tasks.add_task(os.remove, temp_in.name)
  background_tasks.add_task(os.remove, temp_out.name)
2 filename = os.path.basename(temp_out.name)
3 return {
4     "transcript": transcribed_text,
5     "ai_answer": ai_response,
6     "audio_filename": filename
7 }
```

Votre endpoint final devrait être similaire à ceci.

```
# -- Endpoint Voix

@app.post("/voice-chat")
async def voice_chat(
    audio: UploadFile = File(...),
    background_tasks: BackgroundTasks = None
):

    if USE_DEMO_VOICE:
        # --- Mode démo rapide ---
        demo_transcript = "Bonjour, ceci est une transcription simulée."
        demo_ai_answer = "Voici une réponse IA simulée par le mode démo."
        demo_audio_filename = "demo.wav"

        # Simule un fichier audio demo
        demo_path = os.path.join(tempfile.gettempdir(), demo_audio_filename)
        with open(demo_path, "wb") as f:
            f.write(b"\x00\x00") # Écrit un mini fichier wav vide (ça évite l'erreur 404)

        return {
            "transcript": demo_transcript,
            "ai_answer": demo_ai_answer,
            "audio_filename": demo_audio_filename
        }

    # --- Pipeline réelle ---
    # 1. Enregistre l'audio temporaire
    temp_in = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
    content = await audio.read()
    temp_in.write(content)
    temp_in.flush()
    temp_in.close()

    # 2. Transcrit l'audio en texte
    transcribed_text = transcribe_audio(temp_in.name)

    # 3. Passe ce texte à l'IA pour obtenir la réponse IA
    prompt = f"L'utilisateur dit : {transcribed_text}. Que recommandes-tu comme vin ?"
    ai_response = await run_in_threadpool(llm_client.complete, prompt)

    # 4. Synthétise la réponse en audio
    temp_out = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
    speak_text(ai_response, temp_out.name)
    temp_out.close()

    # 5. Nettoyage
    if background_tasks is not None:
        background_tasks.add_task(os.remove, temp_in.name)
        background_tasks.add_task(os.remove, temp_out.name)

    filename = os.path.basename(temp_out.name)

    return {
        "transcript": transcribed_text,
        "ai_answer": ai_response,
        "audio_filename": filename
    }
```

```
1 @app.post("/voice-chat")
2 async def voice_chat(
3     audio: UploadFile = File(...),
4     background_tasks: BackgroundTasks = None
5 ):
6
7 if USE_DEMO_VOICE:
8     # --- Mode démo rapide ---
9     demo_transcript = "Bonjour, ceci est une transcription simulée."
10    demo_ai_answer = "Voici une réponse IA simulée par le mode démo."
11    demo_audio_filename = "demo.wav"
12
```

```
13 # Simule un fichier audio demo
14 demo_path = os.path.join(tempfile.gettempdir(), demo_audio_filename)
15 with open(demo_path, "wb") as f:
16     f.write(b"\x00\x00") # Écrit un mini fichier wav vide (ça évite l'erreur 404)
17
18 return {
19     "transcript": demo_transcript,
20     "ai_answer": demo_ai_answer,
21     "audio_filename": demo_audio_filename
22 }
23
24 # --- Pipeline réelle ---
25 # 1. Enregistre l'audio temporaire
26 temp_in = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
27 content = await audio.read()
28 temp_in.write(content)
29 temp_in.flush()
30 temp_in.close()
31
32 # 2. Transcrit l'audio en texte
33 transcribed_text = transcribe_audio(temp_in.name)
34
35 # 3. Passe ce texte à l'IA pour obtenir la réponse IA
36 prompt = f"L'utilisateur dit : {transcribed_text}. Que recommandes-tu comme vin ?"
37 ai_response = await run_in_threadpool(llm_client.complete, prompt)
38
39 # 4. Synthétise la réponse en audio
40 temp_out = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
41 speak_text(ai_response, temp_out.name)
42 temp_out.close()
43
44 # 5. Nettoyage
45 if background_tasks is not None:
46     background_tasks.add_task(os.remove, temp_in.name)
47     background_tasks.add_task(os.remove, temp_out.name)
48
49 filename = os.path.basename(temp_out.name)
50
51 return {
52     "transcript": transcribed_text,
53     "ai_answer": ai_response,
54     "audio_filename": filename
55 }
```

Et votre fichier main.py devrait ressembler à cela après avoir intégré convenablement l'endpoint voix.

```

# app = FastAPI() to chat_endpoint
from fastapi import FastAPI
from api.schemas import ChatRequest, ChatResponse
from core.services import get_product_info
from core.llm_client import LLMClient
from starlette.concurrency import run_in_threadpool
from fastapi.middleware.cors import CORSMiddleware

from core.services import analyze_sentiment

from fastapi import FastAPI, UploadFile, File

from core.vision_client import analyze_image

from core.voice_service import USE_DEMO_VOICE, transcribe_audio, speak_text
from fastapi import BackgroundTasks
import tempfile
import os

app = FastAPI()
llm_client = LLMClient()

origins = [
    "http://localhost:5173",
    "http://127.0.0.1:5173"
]

# Endpoint Chat
@app.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    dataset_response = get_product_info(request.user_message)

    if dataset_response:
        recommendation = dataset_response
    else:
        prompt = f"l'utilisateur demande : {request.user_message}. Réponds uniquement en français. Que recommandes-tu comme vin ?"
        recommendation = await run_in_threadpool(llm_client.complete, prompt)

    # --- Analyse de sentiment avancée ---
    sentiment, score = analyze_sentiment(request.user_message)
    escalate = (sentiment == "negative" and score > 0.75)

    # Affichage côté terminal pour debug :
    if escalate:
        print(f"[ESCALADE] Message critique détecté : \'{request.user_message}\'' (sentiment : {sentiment}, score : {score})")
    else:
        print(f"[INFO] Message traité : \'{request.user_message}\'' (sentiment : {sentiment}, score : {score})")

    return ChatResponse(
        recommendation=recommendation,
        sentiment=sentiment,
        escalate_to_human=escalate
    )

# --- Endpoint Image
@app.post("/chat/photo")
async def chat_photo(file: UploadFile = File(...)):
    # 1. Enregistre l'image temporairement sur le disque
    path = f"/tmp/{file.filename}"
    contents = await file.read()
    with open(path, "wb") as f:
        f.write(contents)

    # 2. Analyse l'image
    desc = analyze_image(path)
    return {"description": desc}

# --- Endpoint Voix
@app.post("/voice-chat")
async def voice_chat(
    audio: UploadFile = File(...),
    background_tasks: BackgroundTasks = None
):
    if USE_DEMO_VOICE:
        # --- Mode démo rapide ---
        demo_transcript = "Bonjour, ceci est une transcription simulée."
        demo_ai_answer = "Voici une réponse IA simulée par le mode démo."
        demo_audio_filename = "demo.wav"

        # Simule un fichier audio démo
        demo_path = os.path.join(tempfile.gettempdir(), demo_audio_filename)
        with open(demo_path, "wb") as f:
            f.write(b"1000000") # Écrit un mini fichier wav vide (ça évite l'erreur 484)

        return {
            "transcript": demo_transcript,
            "ai_answer": demo_ai_answer,
            "audio_filename": demo_audio_filename
        }

    # --- Pipeline réelle ---
    # 1. Enregistre l'audio temporaire
    temp_in = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
    content = await audio.read()
    temp_in.write(content)
    temp_in.flush()
    temp_in.close()

    # 2. Transcrit l'audio en texte
    transcribed_text = transcribe_audio(temp_in.name)

    # 3. Passe ce texte à l'IA pour obtenir la réponse IA
    prompt = f"l'utilisateur dit : {transcribed_text}. Que recommandes-tu comme vin ?"
    ai_response = await run_in_threadpool(llm_client.complete, prompt)

    # 4. Synthétise la réponse en audio
    temp_out = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
    speak_text(ai_response, temp_out.name)
    temp_out.close()

    # 5. Nettoyage
    if background_tasks is not None:
        background_tasks.add_task(os.remove, temp_in.name)
        background_tasks.add_task(os.remove, temp_out.name)

    filename = os.path.basename(temp_out.name)

    return {
        "transcript": transcribed_text,
        "ai_answer": ai_response,
        "audio_filename": filename
    }

app.add_middleware(
    CORSMiddleware,
    allow_origins=origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

```

```

1 from fastapi import FastAPI
2 from api.schemas import ChatRequest, ChatResponse
3 from core.services import get_product_info
4 from core.llm_client import LLMClient
5 from starlette.concurrency import run_in_threadpool

```

```

6 from fastapi.middleware.cors import CORSMiddleware
7
8 from core.services import analyze_sentiment
9
10 from fastapi import FastAPI, UploadFile, File
11
12 from core.vision_client import analyze_image
13
14 from core.voice_service import USE_DEMO_VOICE, transcribe_audio, speak_text
15 from fastapi import BackgroundTasks
16 import tempfile
17 import os
18
19
20 app = FastAPI()
21 llm_client = LLMClient()
22
23 origins = [
24 "http://localhost:5173",
25 "http://127.0.0.1:5173"
26 ]
27
28 # Endpoint Chat
29
30 @app.post("/chat", response_model=ChatResponse)
31 async def chat_endpoint(request: ChatRequest):
32     dataset_response = get_product_info(request.user_message)
33
34     if dataset_response:
35         recommendation = dataset_response
36     else:
37         prompt = f"L'utilisateur demande : {request.user_message}. Réponds uniquement en
                 français. Que recommandes-tu comme vin ?"
38         recommendation = await run_in_threadpool(llm_client.complete, prompt)
39
40 # --- Analyse de sentiment avancée ---
41 sentiment, score = analyze_sentiment(request.user_message)
42 escalate = (sentiment == "negative" and score > 0.75)
43
44 # Affichage côté terminal pour debug :
45 if escalate:
46     print(f"[ESCALADE] Message critique détecté : \"{request.user_message}\" (sentiment :
           {sentiment}, score : {score})")
47 else:
48     print(f"[INFO] Message traité : \"{request.user_message}\" (sentiment : {sentiment},
           score : {score})")
49
50
51 return ChatResponse(
52     recommendation=recommendation,
53     sentiment=sentiment,
54     escalate_to_human=escalate
55 )
56
57 # ---Endpoint Image
58
59 @app.post("/chat/photo")
60 async def chat_photo(file: UploadFile = File(...)):
61     # 1 Enregistre l'image temporairement sur le disque
62     path = f"/tmp/{file.filename}"
63     contents = await file.read()
64     with open(path, "wb") as f:
65         f.write(contents)

```

```

66
67 # 2. Analyse l'image
68 desc = analyze_image(path)
69 return {"description": desc}
70
71 # -- Endpoint Voix
72
73 @app.post("/voice-chat")
74 async def voice_chat(
75     audio: UploadFile = File(...),
76     background_tasks: BackgroundTasks = None
77 ):
78
79     if USE_DEMO_VOICE:
80         # --- Mode démo rapide ---
81         demo_transcript = "Bonjour, ceci est une transcription simulée."
82         demo_ai_answer = "Voici une réponse IA simulée par le mode démo."
83         demo_audio_filename = "demo.wav"
84
85         # Simule un fichier audio demo
86         demo_path = os.path.join(tempfile.gettempdir(), demo_audio_filename)
87         with open(demo_path, "wb") as f:
88             f.write(b"\x00\x00") # Écrit un mini fichier wav vide (ça évite l'erreur 404)
89
90         return {
91             "transcript": demo_transcript,
92             "ai_answer": demo_ai_answer,
93             "audio_filename": demo_audio_filename
94         }
95
96     # --- Pipeline réelle --
97     # 1. Enregistre l'audio temporaire
98     temp_in = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
99     content = await audio.read()
100     temp_in.write(content)
101     temp_in.flush()
102     temp_in.close()
103
104     # 2. Transcrit l'audio en texte
105     transcribed_text = transcribe_audio(temp_in.name)
106
107     # 3. Passe ce texte à l'IA pour obtenir la réponse IA
108     prompt = f"L'utilisateur dit : {transcribed_text}. Que recommandes-tu comme vin ?"
109     ai_response = await run_in_threadpool(llm_client.complete, prompt)
110
111     # 4. Synthétise la réponse en audio
112     temp_out = tempfile.NamedTemporaryFile(delete=False, suffix=".wav")
113     speak_text(ai_response, temp_out.name)
114     temp_out.close()
115
116     # 5. Nettoyage
117     if background_tasks is not None:
118         background_tasks.add_task(os.remove, temp_in.name)
119         background_tasks.add_task(os.remove, temp_out.name)
120
121     filename = os.path.basename(temp_out.name)
122
123     return {
124         "transcript": transcribed_text,
125         "ai_answer": ai_response,
126         "audio_filename": filename
127     }
128

```

```
129
130 app.add_middleware(
131     CORSMiddleware,
132     allow_origins=origins,
133     allow_credentials=True,
134     allow_methods=["*"],
135     allow_headers=["*"],
136 )
```

Passons au test.

Pour ce faire, enregistrez un audio en anglais de préférence, sinon la transcription ne fonctionnera pas correctement. L'audio de quelques secondes (5 secondes suffiront) devra être au format .wav sinon, le frontend retournera une erreur.

Une fois fait, rendez-vous dans votre terminal,

Activez uniquement le mode démo voice, et désactivez toutes les autres IA (texte, sentiment, image).

Puis lancez votre serveur backend.


```
demo@dev USE_DEMO_TEXT=1 USE_DEMO_SENTIMENT=1 USE_DEMO_IMAGE=1 USE_DEMO_VOICE=0 uvicorn api.main:app --reload
```

```
1 USE_DEMO_TEXT=1 USE_DEMO_SENTIMENT=1 USE_DEMO_IMAGE=1 USE_DEMO_VOICE=0 uvicorn  
api.main:app --reload
```

Lancez votre serveur côté frontend.

```
demo@dev npm run dev
```

```
1 npm run dev
```

Depuis le chatbot, cliquez sur l'icône micro et téléchargez votre audio. Cliquez ensuite sur le bouton d'envoi.

Conciergerie Bon Vivant 🛎



Le temps de chargement est variable. Il peut être égal à plusieurs dizaines de minutes.
Si vous obtenez une recommandation basée votre audio, ça fonctionne.

Conciergerie Bon Vivant 🛎



 Remarque

La phrase prononcée dans notre test était "*what kind of cheese with red wine?*".

Nous avons donc désormais un chatbot multimodal parfaitement fonctionnel. Vous pouvez être fier du chemin parcouru jusqu'ici.

IV Synthèse

A. Flashcards

Chatbot multimodal

Chatbot capable d'interagir via texte, image et voix pour une meilleure expérience.

Analyse de sentiment

Technique IA détectant l'humeur d'un texte pour adapter la réponse ou escalade.

Pipeline HuggingFace

Composant IA tout-en-un pour exécuter des tâches NLP avec modèles pré-entraînés.

Lazy loading

Chargement d'un composant uniquement lors de son premier usage pour optimiser ressources.

Vosk

Modèle de reconnaissance vocale convertissant la parole en texte pour traitement IA.

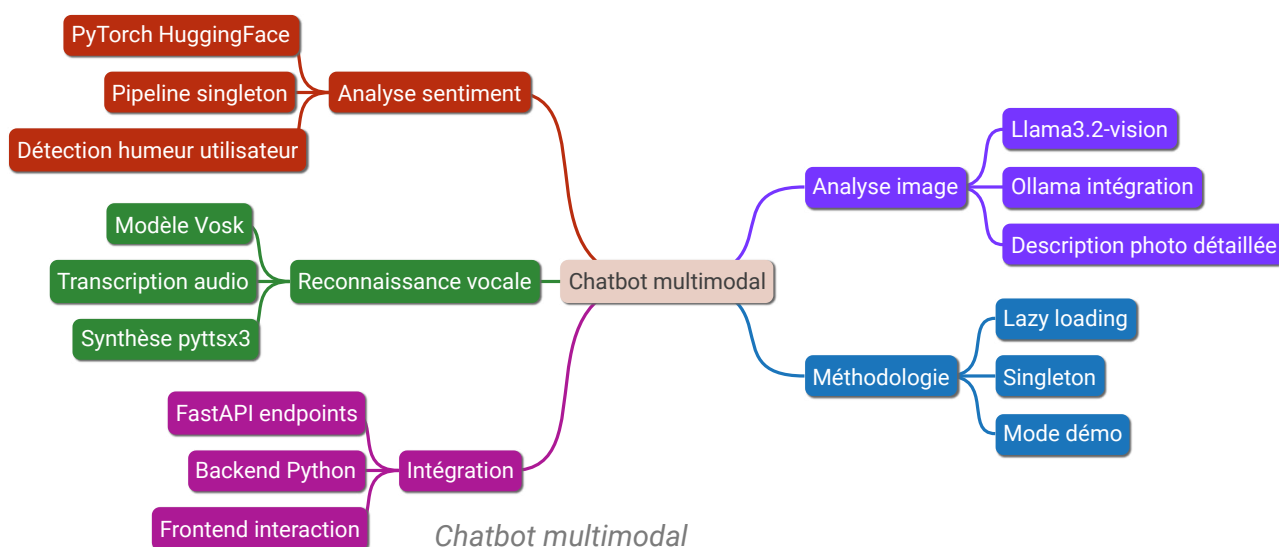
Synthèse vocale pyttsx3

Bibliothèque Python transformant du texte en voix synthétique pour réponses audio.

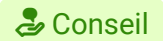
La théorie en Bref

🔗 Fondamental

Un **chatbot multimodal** permet d'interagir via plusieurs modes : texte, image et voix. L'**analyse de sentiment** utilise un modèle PyTorch via HuggingFace pour détecter l'humeur de l'utilisateur et déclencher une **escalade humaine** si nécessaire. L'**analyse d'image** est réalisée avec le modèle Llama3.2-vision via Ollama, offrant une description détaillée des photos. La reconnaissance vocale repose sur le modèle **Vosk** pour convertir la parole en texte, et la synthèse vocale utilise **pyttsx3** pour générer des réponses audio. Ces fonctionnalités enrichissent l'expérience utilisateur en rendant le chatbot plus accessible et interactif.



Les conseils pro



Comment intégrer l'analyse de sentiment dans un chatbot ?

- Utiliser un pipeline HuggingFace avec lazy loading.
- Détecter les sentiments négatifs pour escalade humaine.
- Mettre à jour régulièrement les modèles.
- Tester avec des messages variés.

Comment gérer l'analyse d'image dans une application web ?

- Enregistrer temporairement les images côté serveur.
- Utiliser un modèle IA spécialisé comme Llama3.2-vision.
- Retourner une description claire au frontend.
- Prévoir un mode démo pour tests rapides.

Comment implémenter la reconnaissance vocale et synthèse audio ?

- Choisir un modèle léger et performant comme Vosk.
- Transcrire l'audio en texte avant traitement IA.
- Utiliser pyttsx3 pour générer la réponse vocale.
- Gérer proprement les fichiers temporaires.

Comment assurer la robustesse des endpoints multimodaux ?

- Activer des modes démo pour tests sans IA réelle.
- Gérer les erreurs liées aux formats de fichiers.
- Utiliser des middlewares CORS adaptés.
- Surveiller les logs pour détecter les escalades.

V Auto-évaluation

A. Exercice : Quiz

[solution n°1 p. 67]

Exercice

Pourquoi utilise-t-on PyTorch avec HuggingFace pour l'analyse de sentiment dans ce chatbot ?

Parce que PyTorch est le seul framework compatible avec HuggingFace.

Parce que PyTorch permet d'utiliser un pipeline reconnu dans l'écosystème IA moderne et HuggingFace propose des modèles pré-entraînés efficaces.

Parce que PyTorch est plus rapide que tous les autres frameworks.

Parce que HuggingFace ne fonctionne qu'avec PyTorch.

Exercice

Quels sont les avantages combinés du lazy loading et du pattern singleton dans l'intégration du pipeline HuggingFace ?

A Économiser la mémoire en chargeant le modèle uniquement à l'utilisation.

B Créer plusieurs instances du pipeline pour gérer la charge.

C Réduire le temps de démarrage de l'application.

D Garantir une seule instance du pipeline pour toute l'application.

Exercice

Le chatbot escalade automatiquement un message à un humain si le sentiment est négatif avec un score supérieur à 0.75. Vrai ou Faux ?

Vrai

Faux

Exercice

Quel modèle est utilisé pour l'analyse d'image dans ce chatbot multimodal ?

Llama3.2-vision via Ollama

PyTorch avec HuggingFace

Vosk

pyttsx3

Exercice

Pourquoi l'image est-elle enregistrée temporairement sur le disque dans l'endpoint d'analyse d'image ?

Pour permettre au modèle IA de la lire depuis un chemin accessible.

Pour conserver l'image en base de données.

Pour compresser l'image avant analyse.

Pour envoyer l'image directement au frontend.

Exercice

Quel modèle est utilisé pour la reconnaissance vocale dans ce chatbot ?

Vosk

Llama3.2-vision

HuggingFace pipeline

pyttsx3

Exercice

Quelle bibliothèque est utilisée pour la synthèse vocale (Text-to-Speech) dans ce projet ?

pyttsx3

Vosk

HuggingFace

Ollama

Exercice

Quelles sont les étapes principales du traitement dans l'endpoint voix ?

- A** Enregistrer l'audio temporairement sur le disque.
- B** Transcrire l'audio en texte avec Vosk.
- C** Passer le texte à l'IA pour obtenir une réponse.
- D** Synthétiser la réponse en audio avec pyttsx3.
- E** Envoyer directement l'audio au frontend sans traitement.

Exercice

Le mode démo permet de tester les endpoints sans charger les modèles IA réels. Vrai ou Faux ?

Vrai

Faux

Exercice

Pourquoi utilise-t-on un middleware CORS dans ce projet ?

- Pour autoriser les requêtes entre le frontend et backend sur différentes origines.
- Pour sécuriser les données utilisateurs.
- Pour accélérer le traitement des requêtes.
- Pour gérer les fichiers temporaires.

Exercice

Associez les termes à leur définition correcte.

- A** Pipeline HuggingFace
- B** Singleton
- C** Vosk
- D** Lazy loading

Chargement d'un composant uniquement lors de son premier usage.	Création d'une seule instance d'un objet réutilisée dans toute l'application.	Composant tout-en-un pour exécuter des tâches IA avec modèles pré-entraînés.	Modèle de reconnaissance vocale convertissant la parole en texte.
---	---	--	---

Exercice

Placez dans l'ordre correct les étapes de l'endpoint d'analyse de sentiment.

A Analyse du texte utilisateur avec le pipeline

B Chargement du pipeline HuggingFace (lazy loading et singleton)

C Détection d'escalade selon sentiment et score

D Retour du label et score de sentiment

Réponse : ____

Exercice

Ordonnez les étapes du traitement dans l'endpoint voix.

A Synthèse vocale de la réponse

B Obtention de la réponse IA à partir du texte

C Nettoyage des fichiers temporaires

D Transcription de l'audio en texte avec Vosk

E Enregistrement temporaire du fichier audio

Réponse : ____

Exercice

Ordonnez les étapes de l'endpoint d'analyse d'image.

A Appel à la fonction d'analyse d'image

B Enregistrement temporaire de l'image sur disque

C Retour de la description générée au frontend

Réponse : ____

Exercice

Quels critères déclenchent l'escalade vers un humain dans l'analyse de sentiment ?

A Sentiment négatif

B Score supérieur à 0.75

C Sentiment neutre

D Score inférieur à 0.5

Exercice

Quels sont les bénéfices d'un chatbot multimodal intégrant texte, image et voix ?

A Accessibilité améliorée

B Interaction plus riche et naturelle

C Réduction des coûts de développement

D Meilleure expérience utilisateur

B. Exercice : Cas pratique

Question

[solution n°2 p. 72]

Vous êtes chef de projet IA dans un grand groupe. Lors d'une réunion, vous devez proposer la création d'un chatbot multimodal et devrez expliquer les fonctionnalités de votre chatbot ainsi que la valeur métier.

Solutions des exercices

Solution n°1

[exercice p. 62]

Exercice

Pourquoi utilise-t-on PyTorch avec HuggingFace pour l'analyse de sentiment dans ce chatbot ?

Parce que PyTorch est le seul framework compatible avec HuggingFace.

✓ Parce que PyTorch permet d'utiliser un pipeline reconnu dans l'écosystème IA moderne et HuggingFace propose des modèles pré-entraînés efficaces.

Parce que PyTorch est plus rapide que tous les autres frameworks.

Parce que HuggingFace ne fonctionne qu'avec PyTorch.

🔍 PyTorch est choisi car il permet d'utiliser un pipeline reconnu dans l'écosystème IA moderne, et HuggingFace offre des modèles pré-entraînés efficaces et simples à intégrer, ce qui facilite l'analyse de sentiment avancée.

Exercice

Quels sont les avantages combinés du lazy loading et du pattern singleton dans l'intégration du pipeline HuggingFace ?

A Économiser la mémoire en chargeant le modèle uniquement à l'utilisation.

B Créer plusieurs instances du pipeline pour gérer la charge.

C Réduire le temps de démarrage de l'application.

D Garantir une seule instance du pipeline pour toute l'application.

🔍 Le lazy loading permet de charger le modèle uniquement lorsqu'il est utilisé, économisant ainsi mémoire et temps de démarrage. Le pattern singleton garantit qu'une seule instance du pipeline est créée et réutilisée, évitant des charges inutiles.

Exercice

Le chatbot escalade automatiquement un message à un humain si le sentiment est négatif avec un score supérieur à 0.75. Vrai ou Faux ?

✓ Vrai

Faux



Le chatbot détecte un sentiment négatif avec un score supérieur à 0.75 et déclenche une escalade vers un humain pour gérer la situation critique.

Exercice

Quel modèle est utilisé pour l'analyse d'image dans ce chatbot multimodal ?

✓ Llama3.2-vision via Ollama

PyTorch avec HuggingFace

Vosk

pyttsx3



Le modèle Llama3.2-vision via Ollama est utilisé pour analyser les images et fournir une description détaillée dans le chatbot multimodal.

Exercice

Pourquoi l'image est-elle enregistrée temporairement sur le disque dans l'endpoint d'analyse d'image ?

✓ Pour permettre au modèle IA de la lire depuis un chemin accessible.

Pour conserver l'image en base de données.

Pour compresser l'image avant analyse.

Pour envoyer l'image directement au frontend.



L'image est enregistrée temporairement sur le disque pour que le modèle IA puisse y accéder via un chemin local, facilitant ainsi son analyse.

Exercice

Quel modèle est utilisé pour la reconnaissance vocale dans ce chatbot ?

✓ Vosk

Llama3.2-vision

HuggingFace pipeline

pyttsx3

🔍 Le modèle Vosk est utilisé pour la reconnaissance vocale (Speech-to-Text) dans ce chatbot multimodal.

Exercice

Quelle bibliothèque est utilisée pour la synthèse vocale (Text-to-Speech) dans ce projet ?

✓ pyttsx3

Vosk

HuggingFace

Ollama

🔍 La bibliothèque pyttsx3 est utilisée pour transformer le texte en voix synthétique, permettant de générer des réponses audio.

Exercice

Quelles sont les étapes principales du traitement dans l'endpoint voix ?

A Enregistrer l'audio temporairement sur le disque.

B Transcrire l'audio en texte avec Vosk.

C Passer le texte à l'IA pour obtenir une réponse.

D Synthétiser la réponse en audio avec pyttsx3.

E Envoyer directement l'audio au frontend sans traitement.

🔍 L'endpoint voix enregistre d'abord l'audio, le transcrit en texte via Vosk, obtient une réponse IA, puis synthétise cette réponse en audio avant de la retourner.

Exercice

Le mode démo permet de tester les endpoints sans charger les modèles IA réels. Vrai ou Faux ?

✓ Vrai

Faux



Le mode démo simule les réponses pour éviter de charger les modèles IA réels, ce qui facilite les tests rapides et évite les blocages.

Exercice

Pourquoi utilise-t-on un middleware CORS dans ce projet ?

✓ Pour autoriser les requêtes entre le frontend et backend sur différentes origines.

Pour sécuriser les données utilisateurs.

Pour accélérer le traitement des requêtes.

Pour gérer les fichiers temporaires.



Le middleware CORS est utilisé pour permettre au frontend et backend de communiquer même s'ils sont sur des origines différentes, évitant ainsi les blocages liés à la politique de même origine.

Exercice

Associez les termes à leur définition correcte.

Chargement d'un composant uniquement lors de son premier usage.	Création d'une seule instance d'un objet réutilisée dans toute l'application.	Composant tout-en-un pour exécuter des tâches IA avec modèles pré-entraînés.	Modèle de reconnaissance vocale convertissant la parole en texte.
Lazy loading	Singleton	Pipeline HuggingFace	Vosk



Chaque terme correspond à une notion clé dans l'implémentation du chatbot multimodal, facilitant la gestion des ressources et les fonctionnalités IA.

Exercice

Placez dans l'ordre correct les étapes de l'endpoint d'analyse de sentiment.

A Chargement du pipeline HuggingFace (lazy loading et singleton)

B Analyse du texte utilisateur avec le pipeline

C Retour du label et score de sentiment

D Détection d'escalade selon sentiment et score

🔍 L'endpoint charge d'abord le pipeline, analyse le texte, récupère le résultat puis décide d'escalader ou non selon le score.

Exercice

Ordonnez les étapes du traitement dans l'endpoint voix.

A Enregistrement temporaire du fichier audio

B Transcription de l'audio en texte avec Vosk

C Obtention de la réponse IA à partir du texte

D Synthèse vocale de la réponse

E Nettoyage des fichiers temporaires

🔍 Le traitement voix suit ces étapes pour convertir l'audio en texte, générer une réponse et la transformer en audio.

Exercice

Ordonnez les étapes de l'endpoint d'analyse d'image.

A Enregistrement temporaire de l'image sur disque

B Appel à la fonction d'analyse d'image

C Retour de la description générée au frontend

🔍 L'image est d'abord sauvegardée, analysée par le modèle IA, puis la description est renvoyée au frontend.

Exercice

Quels critères déclenchent l'escalade vers un humain dans l'analyse de sentiment ?

A Sentiment négatif

B Score supérieur à 0.75

C Sentiment neutre

D Score inférieur à 0.5



L'escalade est déclenchée uniquement si le sentiment est négatif et que le score de confiance est supérieur à 0.75.

Exercice

Quels sont les bénéfices d'un chatbot multimodal intégrant texte, image et voix ?

A

Accessibilité améliorée

B

Interaction plus riche et naturelle

C

Réduction des coûts de développement

D

Meilleure expérience utilisateur



Un chatbot multimodal offre une meilleure accessibilité, des interactions plus riches et une expérience utilisateur améliorée, mais pas nécessairement une réduction des coûts.

[exercice p. 66]

Solution n°2

Avant, il y avait Google. Aujourd'hui, il y a chatGPT.

Plutôt similaire dans le fond, chatGPT a un avantage concurrentiel énorme comparé à Google.

En effet, quand l'un permet d'effectuer des requêtes basiques dans une grande base de données, ChatGPT permet d'interagir directement avec l'information.

On passe d'une requête basique à une requête intelligente.

C'est ce que nous allons tenter d'implémenter sur le site internet du groupe.

Avant, la plupart des utilisateurs partaient à la recherche des informations en naviguant de page en page.

Aujourd'hui, les informations pertinentes iront au contact de l'utilisateur en plaçant un chatbot multimodal au centre de l'expérience utilisateur.

Très concrètement, voici comment celui-ci va s'articuler.

Il comprendra plusieurs fonctions : l'image, la voix et le texte.

L'image permettra d'effectuer des recherches en photographiant un produit de notre marque. La voix, quant à elle, permettra à l'utilisateur d'interagir avec notre chatbot de manière naturelle. Cela est également un gros avantage en termes d'accessibilité.

Enfin, la fonction textuelle permettra de poser des questions via un champ de texte.

À ces 3 fonctionnalités principales, nous allons ajouter en sous-couche une fonction d'analyse de sentiment poussée afin de récolter des informations précieuses concernant les avis clients, mais aussi déclencher un bouton "Parler à un humain" si la situation l'exige.

En faisant du chatbot le centre de notre site internet, nous donnons une image moderne de l'entreprise, parfaitement en accord avec les besoins actuels du marché.